

O objetivo deste trabalho consiste em implementar uma estrutura completa de Árvore B (de grau m) residente em memória secundária, integrando as operações de busca (mSearch), inserção (insertB) e remoção (deleteB). A Árvore B deve indexar um campo chave (inteiro). Cada nós deve ter o formato: $n, A_0, (K_1, A_1), \dots, (K_n, A_n)$, conforme descrito em aula.

Requisitos Obrigatórios: Estes tópicos formam a base do trabalho e são exigidos para a obtenção da nota:

- **Estrutura e Paradigma:**
 - O código deve utilizar tipos abstratos de dados, sendo obrigatoriamente implementado usando classes em C++.
 - A árvore deve residir em memória secundária (disco).
 - A raiz não deve ser implementada como uma variável global e deve ser armazenada em disco entre as execuções.
- **Métodos:**
 - Implementar os métodos mSearch, insertB e deleteB.
 - Todos os métodos devem ser implementados de forma iterativa, sendo penalizado o uso de chamadas recursivas.
 - Os métodos de inserção e remoção (insertB e deleteB) devem obrigatoriamente realizar chamadas ao método mSearch para operar.
 - O método mSearch deve retornar os valores processados por meio de uma estrutura de dados apropriada (como uma tupla de valores) e não deve realizar a impressão no terminal (via cout) de dentro do método.
 - O método mSearch deve realizar o empilhamento estruturado dos nós visitados durante a busca, para evitar que os métodos insertB e deleteB necessitem realizar leituras de disco adicionais apenas para encontrar o nó pai.
 - Documentação rigorosa no código: cada método deve possuir comentários descrevendo claramente os parâmetros, funcionalidades e suas pré- e pós-condições.
- **Gerenciamento de Disco (Eficiência):**
 - Por questões de eficiência, o arquivo não deve ser aberto e fechado a cada leitura ou escrita de um nó; a abordagem ideal é manter o arquivo aberto durante as operações.
 - Como o nó da árvore é um registro de tamanho fixo, a leitura e a escrita em disco devem ocorrer em um único comando para a struct inteira (não se deve realizar leituras/escritas fragmentadas para ler chaves ou variáveis uma a uma).
 - A ordem da árvore (m) não deve ser armazenada ocupando espaço dentro de cada nó em disco; esta informação deve estar em um registro ou cabeçalho separado (junto com a raiz, por exemplo).
- **Interface de Uso:**
 - O programa deve conter um menu interativo e em *loop* que permita ao usuário continuar operando até decidir sair.
 - Funcionalidades do menu: Imprimir o índice (árvore B), **imprimir o conteúdo do arquivo principal**, buscar elemento (exibindo a tupla de valores), inserir elemento e remover elemento.
 - Apresentar mensagens de erro apropriadas de acordo com as situações de exceção.

Requisitos Opcionais: estes tópicos podem ser exigidos pelo professor para compor a nota integral de 10, ou tratados como requisitos avançados (novidade/bônus):

- **Arquivo Principal:** A árvore B deve indexar um campo chave de um arquivo (principal) que contém os dados principais sobre uma determinada aplicação prática a ser escolhida pelo grupo (por exemplo: pacientes, funcionários, genes, inventário patrimonial, etc.).
- **Parametrização Dinâmica:** Permitir que o usuário defina a ordem m no início da execução da aplicação, ao invés de defini-la através de uma constante simbólica fixa ou estática (ex: $m=3$ fixo no código). O uso de vetores flexíveis para chaves evita manutenção em código existente.
- **Estatísticas de Acesso:** Implementar um mecanismo de contagem para rastrear a quantidade de leituras e escritas feitas em disco durante a execução das buscas, inserções e remoções.
- **Coleta de Lixo em Disco (Garbage Collection):** Implementar lógica para reaproveitar o espaço em disco gerado pelos nós que forem removidos, garantindo o gerenciamento otimizado de armazenamento no arquivo binário.
- **Análise Empírica:** Realizar comparação e emissão de relatórios de resultados de eficiência do código rodando sob diferentes valores estipulados para a ordem m .

Entrega e Submissão:

- O trabalho pode ser desenvolvido em grupo de até três alunos (sem exceções); um único trabalho deve ser entregue pelo grupo.
- Os nomes dos integrantes do grupo com seus respectivos números USP devem estar presentes em todos os arquivos-fonte, na forma de comentário.
- Todos os arquivos desenvolvidos devem ser entregues em um **único** arquivo de formato ZIP no e-Disciplinas, na respectiva atividade, incluindo a apresentação.

Apresentação: cada grupo terá 20 minutos para a apresentação do trabalho, o ideal é focar nos aspectos arquiteturais, nas decisões de projeto (especialmente no gerenciamento de disco) e em como o grupo resolveu os desafios do algoritmo, evitando mostrar "paredes de código". Aqui está uma sugestão de *template* estruturado para os slides, dividido pelo tempo estimado para garantir que o grupo cubra todos os requisitos essenciais sem ultrapassar o limite.

Template de Apresentação: Implementação de Árvores B em Disco

Tempo total sugerido: 15 a 17 minutos de fala + 3 a 5 minutos para perguntas do professor/turma.

Slide 1: Capa (1 min)

- Título do Trabalho:** Implementação de Árvores B em Memória Secundária.
- Disciplina e Instituição:** Algoritmos e Estruturas de Dados II
- Integrantes:** Nome e Número USP de todos (até 3 alunos).

Slide 2: Contextualização e Aplicação (2 min, se aplicável)

- O Domínio Escolhido:** Qual foi a aplicação prática escolhida pelo grupo para indexar? (ex: Registro de Pacientes, Inventário de Loja, Catálogo de Livros).
- A Estrutura do Arquivo Principal:** O que compõe o registro de dados principal (fora a árvore)?
- O Papel da Árvore B:** Explicar brevemente qual chave está sendo indexada e como ela aponta para o arquivo de dados.

Slide 3: Arquitetura e Estruturas de Dados (2 min)

- Modelagem Orientada a Objetos:** Quais classes foram criadas? (ex: NoArvoreB, ArvoreB, GerenciadorDisco).
- Estrutura do Nó:** Como o nó foi desenhado para ter um tamanho fixo (n, chaves, ponteiros)?
- Boas práticas adotadas:** Destacar o encapsulamento e justificar onde informações globais (como a Ordem m e a localização da Raiz) foram armazenadas para evitar o uso de variáveis globais.

Slide 4: O Coração do Trabalho: Gerenciamento de I/O em Disco (3 min)

(Dica: Este é um dos pontos mais avaliados pelo professor. O grupo deve demonstrar que entendeu a diferença entre memória principal e secundária).

- Estratégia de Leitura/Escrita:** Como o grupo evitou o *overhead* do Sistema Operacional (explicar que o arquivo é mantido aberto e manipulado via *offsets*).
- Transferência em Bloco:** Mostrar (visualmente ou com um breve snippet) como a leitura/escrita transfere a struct inteira de uma só vez.
- Reaproveitamento de Nós (Garbage Collection - Opcional/Bônus):** Se o grupo implementou, explicar como os nós excluídos no disco são reaproveitados.

Slide 5: O Método mSearch (Busca Iterativa) (3 min)

- Abordagem Iterativa:** Como a busca desce na árvore sem usar recursão.
- O Retorno:** Mostrar que o método retorna uma estrutura encapsulada (a tripla solicitada) e não usa prints (cout) internos.
- A Pilha de Caminho (Empilhamento):** Explicar como o algoritmo armazena os nós visitados (o caminho da raiz até a folha) na memória principal. Demonstrar por que isso é crucial para tornar a Inserção e a Remoção eficientes (evitando reler o disco para achar o pai).

Slide 6: Inserção (insertB) (3 min)

- Fluxo de Inserção:** Diagrama simples mostrando: Chamada do mSearch => Tentativa de inserir na folha => *Split* (cisão) se necessário => Propagação para o pai usando a pilha.
- Tratamento da Raiz:** O que acontece quando a raiz sofre *split* e a árvore cresce em altura.

Slide 7: Remoção (deleteB) (3 min)

- Fluxo de Remoção:** Fluxograma mostrando: Chamada do mSearch => Remoção da folha (ou troca com sucessor, se for nó interno) => Verificação de *underflow*.
- Rebalanceamento:** Como o grupo lidou com o empréstimo (redistribuição) de chaves de irmãos adjacentes ou a concatenação (fusão) de nós.

Slide 8: Resultados Empíricos e Desafios (2 min, se aplicável)

- **Estatísticas de Acesso:** Gráfico ou tabela mostrando a contagem de leituras e escritas no disco variando o tamanho de m (ex: comparar $m=3$, $m=5$, $m=10$). O que acontece com os acessos ao disco quando a ordem aumenta?
- **Desafios do Projeto:** Qual foi a parte mais difícil (ex: debugar a concatenação, lidar com ponteiros de arquivo em C++) e como o grupo resolveu.

Slide 9: Demonstração Rápida / Conclusão (1 min)

- **Screenshot do Menu:** Uma imagem rápida do terminal mostrando a interface amigável construída.
- **Conclusão:** Principais aprendizados sobre o custo de acessar disco vs. memória.

Dicas para o Grupo durante a Apresentação:

1. **Evite print de código extenso:** Em vez de colar 50 linhas de C++, faça diagramas de blocos (caixas representando nós) mostrando como o *split* ou a leitura do arquivo funciona.
2. **Foco no "Como" e "Por que":** O professor já conhece a teoria da Árvore B. O objetivo é mostrar **como** vocês implementaram e **por que** tomaram certas decisões (principalmente focando nas restrições de recursão, variáveis globais e custo do Sistema Operacional).
3. **Pré e Pós-condições:** Mencionem rapidamente durante a explicação dos métodos que o código fonte contém a documentação das pré e pós-condições, conforme exigido.

Na nota do trabalho também serão considerados os seguintes critérios:

- **Arquitetura, Interface e Encapsulamento:** Qualidade do código, uso adequado de C++ (classes, arquivos .h e .cpp), clareza da interface de usuário, tolerância a erros no menu de interação.
- **Método mSearch:** Corretude do algoritmo iterativo, busca no índice, (extra: acesso ao dado no arquivo principal) e empilhamento de nós percorrido.
- **Método insertB:** Corretude da lógica de *split*, propagação e correta alocação de registros secundários e principais, utilizando mSearch.
- **Método deleteB:** Corretude no tratamento de concatenação, redistribuição de nós, utilizando mSearch.
- **Eficiência e Otimização em Disco:** Minimização do *overhead* do Sistema Operacional nas leituras e escritas (abertura contínua do arquivo, cálculos baseados em *offsets* por meio de métodos para esta finalidade, leitura unificada por registros de tamanho fixo).
- **Requisitos Opcionais:** Contadores de acesso a disco, reaproveitamento de nós, parametrização de m , etc.